Guía de pruebas QA

SecurePlan Mobile (Calendario) | Sprint final

Documento	Guía de pruebas QA del sprint final
Dirigido a	José Manuel Mandujano
Repositorio	github.com/HUEJK24/Calendario_Practica
Entorno de prueba	Local (backend + PostgreSQL con Docker)
Versión	1.0
Fecha	5 de noviembre de 2026

La aplicación ya está desplegada, pero estas pruebas se ejecutan en local. Así puedes crear y borrar datos sin tocar la base de producción, ver los logs del servidor y repetir la corrida las veces que haga falta.

Tiempo estimado: 60 a 90 minutos.
Entregable: informe con la tabla de resultados, capturas y los bugs encontrados.

1. Alcance

Estas pruebas cubren la API y la app móvil contra un backend local.

Se prueba	No se prueba
Registro, login y sesión con JWT.	Carga masiva de usuarios o eventos.
CRUD de eventos y aislamiento entre usuarios (IDOR).	Escaneo de penetración o análisis de infraestructura.
Validaciones de entrada y exportación JSON/CSV.	El backend desplegado en Render.
App móvil: login, agenda, creación, edición, borrado y recordatorios.	Compatibilidad con todas las versiones de Android o iOS.

Usa datos inventados. No cargues correos personales, contraseñas reales ni información de terceros. Las cuentas de prueba llevan el prefijo `qa+` y se borran al final.

2. Requisitos previos

Herramienta	Versión o nota
Git	Cualquier versión reciente.
Node.js	18 o superior. Verifica con `node -v`.
Docker Desktop	Debe estar abierto y corriendo antes de iniciar la base.
Postman	Opcional si usas el script; recomendado para el informe.
Expo Go o build de desarrollo	Solo para probar la app contra el backend local.
Navegador	Para abrir los archivos exportados.

Los comandos de esta guía se escriben en macOS o Linux. En Windows con PowerShell cambian las comillas y las rutas, pero los comandos de npm y Docker son los mismos.

3. Preparación del entorno

3.1 Clonar el repositorio

```
git clone https://github.com/HUEJK24/Calendario_Practica.git
cd Calendario_Practica
```

3.2 Instalar dependencias del backend

```
cd backend
npm install
```

3.3 Crear el archivo .env

Copia la plantilla y revisa los valores:

```
cp .env.example .env
```

Variable	Valor para local
PORT	4000
DB_HOST / DB_PORT	localhost / 5432
DB_USER / DB_PASSWORD	calendario / calendario
DB_NAME	calendario
JWT_SECRET	Una cadena larga y aleatoria. No la compartas.
JWT_EXPIRES_IN	1h

El archivo `.env` está en .gitignore. No lo subas al repositorio.

3.4 Levantar PostgreSQL

```
docker compose up -d
docker ps
```

Debe aparecer el contenedor `calendario-db` en estado `Up`. El esquema `sql/init.sql` se aplica solo la primera vez que se crea el volumen. Si la base arrancó vacía, ejecuta `docker compose down -v` y vuelve a levantarla.

3.5 Iniciar la API

```
npm run dev
```

Deja esa terminal abierta. En otra terminal comprueba que responde:

```
curl http://localhost:4000/health
```

La respuesta esperada es `{"status":"ok"}`. Si falla, revisa la sección 9 antes de continuar.

4. Pruebas automatizadas

Ejecuta estas dos y guarda el resultado para el informe.

4.1 Script de humo (35 casos)

```
node qa/scripts/smoke.mjs
```

El script usa solo Node, no instala dependencias. Crea dos usuarios `qa+...@example.com`, recorre todo el flujo y termina con `35/35 pruebas OK`. Si algo falla, lista los casos y sale con código 1.

Bloque	Qué valida
Salud	El endpoint /health responde ok.
Autenticación	Registro, duplicado, clave corta, correo inválido, login, me y tokens inválidos.
Eventos	CRUD, campos extra, orden de horas, fecha, categoría e ids inválidos.
IDOR	Un segundo usuario recibe 404 al leer, editar o borrar un evento ajeno.
Exportación	JSON y CSV con solo los eventos del dueño, formato inválido y sin token.
Cierre	Borrado 204, segundo borrado 404 y ruta inexistente 404.

Contra otro entorno: `QA_BASE_URL=https://mi-api node qa/scripts/smoke.mjs`

4.2 Colección de Postman

Importa estos dos archivos:

```
qa/postman/SecurePlan-QA.postman_collection.json
qa/postman/SecurePlan-QA.postman_environment.json
```

Selecciona el entorno SecurePlan QA Local y ejecuta las carpetas en orden: Salud, Autenticación, Eventos, Exportación, Seguridad.

La colección guarda el token, el correo y el id del evento después de cada petición. Para el caso de IDOR necesitas un segundo usuario: regístralo en Postman, crea un evento con su token y pega el id en la variable `otherEventId`. La petición debe responder 404.

Sin abrir Postman:

```
npx newman run qa/postman/SecurePlan-QA.postman_collection.json \
  -e qa/postman/SecurePlan-QA.postman_environment.json
```

5. Pruebas manuales

Hazlas con Postman, Insomnia o curl. Anota el resultado de cada una en el informe. La columna Resultado esperado indica el código HTTP y el efecto.

ID	Caso	Cómo	Resultado esperado
A01	Registro válido	POST /api/auth/register con correo nuevo y clave de 8 o más.	201, token y usuario sin password.
A02	Registro duplicado	Repite el mismo correo.	409.

ID	Caso	Cómo	Resultado esperado
A03	Clave corta	Registro con password 123.	400 con lista de errores.
A04	Login correcto	POST /api/auth/login.	200 con token.
A05	Login incorrecto	Misma cuenta con otra clave.	401.
A06	Me sin token	GET /api/auth/me sin header Authorization.	401.
A07	Me con token	GET /api/auth/me con Bearer.	200 con el usuario.
E01	Crear evento	POST /api/events con todos los campos.	201, date AAAA-MM-DD y horas HH:MM.
E02	Campo extra	Envía userId en el cuerpo.	400 por .strict().
E03	Horas invertidas	endTime anterior a startTime.	400.
E04	Fecha inválida	date con formato 01-12-2026.	400.
E05	Categoría inválida	category fiesta.	400.
E06	Listar eventos	GET /api/events con el token del dueño.	200 con solo sus eventos.
E07	Leer evento ajeno	Usuario B pide el id de A.	404.
E08	Editar evento ajeno	Usuario B hace PUT sobre el id de A.	404.
E09	Borrar evento ajeno	Usuario B hace DELETE sobre el id de A.	404.
E10	Borrar evento propio	DELETE con el token del dueño.	204 sin cuerpo.
X01	Export JSON	GET /api/events/export?format=json.	200 application/json con arreglo.
X02	Export CSV	GET /api/events/export?format=csv.	200 text/csv con cabecera.
X03	Formato inválido	format=xml.	400.
S01	Token alterado	Cambia un carácter del token.	401.
S02	Ruta inexistente	GET /api/desconocido.	404.
S03	Texto con SQL	title con '; DROP TABLE events; --.	Se guarda como texto y la tabla sigue.

6. App móvil contra el backend local

El APK publicado apunta al backend de Render. Para probar en local usa Expo Go o un build de desarrollo y pasa la URL de tu equipo.

```
ipconfig getifaddr en0        # macOS: IP de tu red local

cd frontend
npm install
EXPO_PUBLIC_API_URL=http://192.168.1.50:4000 npx expo start
```

Reemplaza la IP por la de tu equipo. El teléfono y la computadora deben estar en la misma red. En el emulador de Android usa `http://10.0.2.2:4000`.

Caso móvil	Resultado esperado
Registro y login	Entra a la agenda y el token queda guardado.
Crear evento	Aparece en el día correcto, sin corrimiento de fecha.
Editar y borrar	Los cambios se reflejan al volver a la agenda.
Recordatorio	La notificación se programa con la hora local.

Caso móvil	Resultado esperado
Cerrar sesión	Vuelve al login y el token se borra del almacenamiento seguro.
Token vencido	La app responde 401 y regresa al login sin mostrar el error crudo.

Si el teléfono no conecta, revisa el firewall del equipo y que la API escuche en la red local. `localhost` dentro del teléfono apunta al teléfono, no a tu computadora.

7. Evidencia y reporte de bugs

Guarda cada captura con el id del caso: `A01-registro.png`, `E07-idor.png`. Adjunta la salida del script y el resultado del Collection Runner.

Campo del bug	Qué escribir
Título	Acción y resultado en una línea.
Severidad	Bloqueante, alta, media o baja.
Entorno	Local, sistema operativo y navegador o dispositivo.
Pasos	Lista numerada que cualquiera pueda repetir.
Esperado	Qué debía pasar.
Obtenido	Qué pasó, con el código HTTP o el mensaje.
Evidencia	Captura, log o salida de consola.

Severidad: bloqueante si impide usar la app; alta si rompe una función principal o expone datos de otro usuario; media si hay un error visible con solución alternativa; baja si es cosmético.

8. Limpieza

```
docker exec -i calendario-db psql -U calendario -d calendario < qa/scripts/cleanup.sql
```

Ese comando borra los usuarios `qa+%@example.com` y, en cascada, sus eventos. Después detén la API con `Ctrl+C` y la base con `docker compose down` (sin `-v`, para conservar los datos).

9. Problemas comunes

Síntoma	Solución
docker ps falla o no responde	Abre Docker Desktop y espera a que el daemon arranque.
EADDRINUSE en el puerto 4000	Cierra el proceso anterior con `lsof -i :4000`.
La API responde 500 al arrancar	Revisa el .env y que PostgreSQL esté arriba.
Todas las rutas dan 401	El token venció. Vuelve a hacer login.
Todas las rutas dan 404	La variable baseUrl apunta a otro host o puerto.
La tabla events no existe	El volumen se creó vacío. Usa docker compose down -v y levanta de nuevo.
El teléfono no conecta	Usa la IP de red del equipo, misma Wi-Fi y firewall abierto.

10. Criterios de aceptación

Marca cada punto antes de entregar el informe.

- [ ] El script de humo termina con 35/35 pruebas OK.

[ ] La colección de Postman corre sin fallos, incluido el caso IDOR.

[ ] Registro y login funcionan desde la app móvil.

[ ] Crear, editar y borrar eventos se refleja en la agenda.

[ ] Un usuario no puede leer, editar ni borrar eventos de otro.

[ ] Las validaciones rechazan fechas, horas y categorías inválidas.

[ ] Las respuestas nunca incluyen el campo password.

[ ] La exportación JSON y CSV abre y contiene solo los eventos del usuario.

[ ] El informe incluye resultados, capturas y bugs con severidad.

11. Entrega

Entrega el informe al responsable del proyecto con la tabla de resultados, las capturas y la lista de bugs. Si no hay bugs bloqueantes, el sprint se acepta.

Responsable QA	José Manuel Mandujano
Fecha de ejecución	
Resultado	Aprobado / Con observaciones
Firma	